

인공지능수학 (2026-1)

9. 분류와 클러스터링 (2)

국립금오공과대학교 컴퓨터공학부
컴퓨터공학전공 / 인공지능공학전공
김 경 수

목차



- I. K-means clustering algorithm
- II. K-means clustering 예제

- ① 클러스터링 알고리즘의 정의를 이해하고, 클러스터링 알고리즘과 분류 알고리즘의 차이점을 지도/비지도 학습의 관점에서 설명할 수 있다.
- ② K-means clustering 알고리즘의 정의와 특성, 수행 과정을 이해하고, 이를 논리적으로 설명할 수 있다.
- ③ K-means clustering 알고리즘의 주요 기능을 직접 구현할 수 있다.

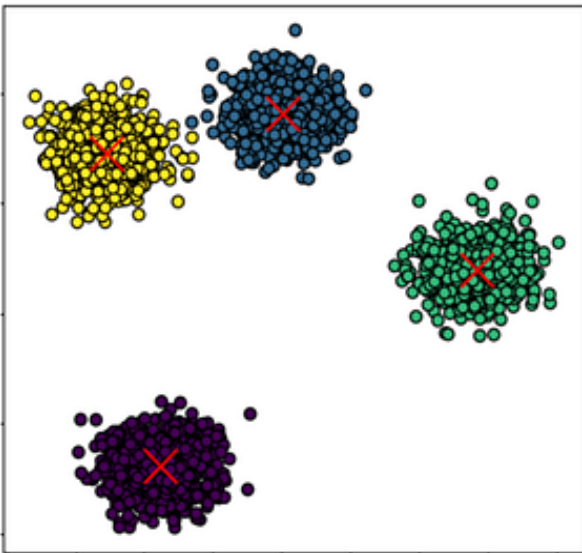
K-Means Clustering 개요

클러스터링 (Clustering) 개요

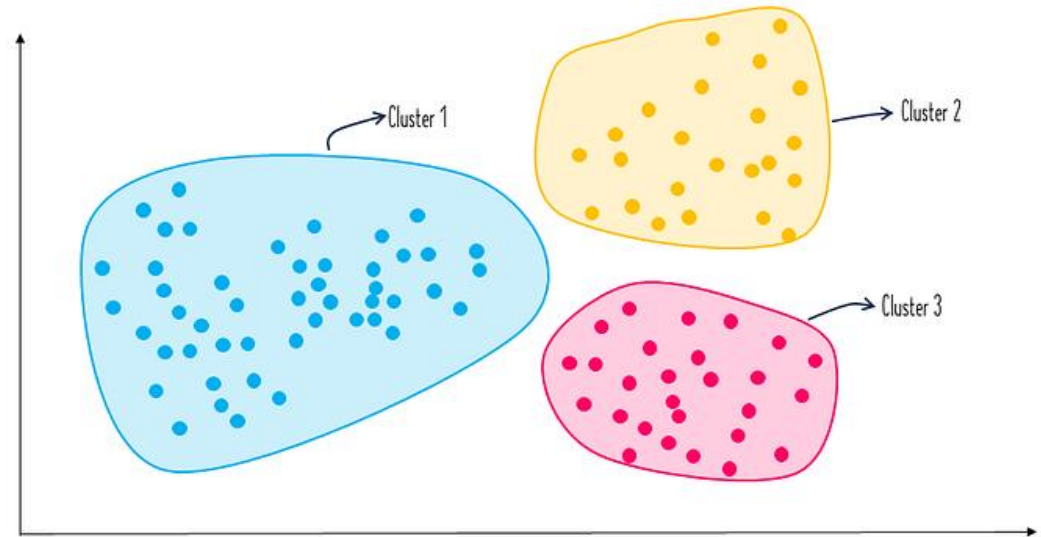
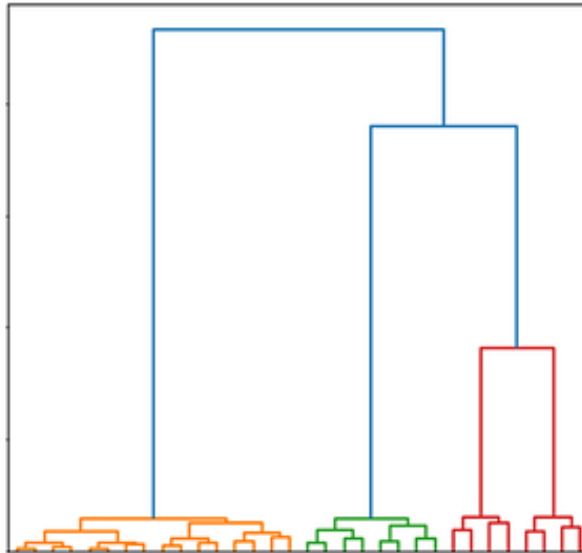
- 클러스터링(clustering)이란 데이터 간의 유사도(similarity) 또는 분포(distribution) 특성을 분석하여 서로 유사성이 높은 데이터끼리 그룹(= cluster)으로 묶는 비지도 학습 방법(unsupervised learning method)
- 분류(classification) 알고리즘과는 달리 사전에 정의된 클래스(class)가 존재하지 않는다. (※ 분류 알고리즘은 지도 학습(supervised learning)에 속하는 방법)
- 클러스터링 알고리즘을 활용하면, 라벨(label)이 주어지지 않는 수많은 데이터 내에서 서로 유사성이 높은 데이터들을 효과적으로 발견할 수 있다.
- 대표적인 클러스터링 알고리즘의 종류
 - K-means clustering algorithm
 - Hierarchical clustering algorithm

클러스터링 알고리즘의 종류

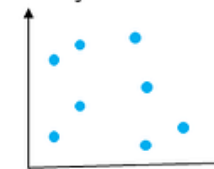
K-Means Clustering



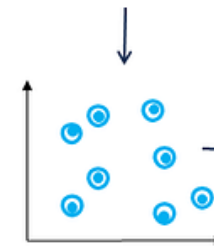
Hierarchical Clustering



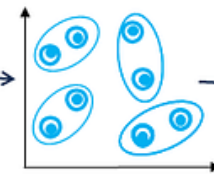
The original dataset



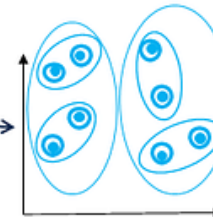
Agglomerative Hierarchical Clustering



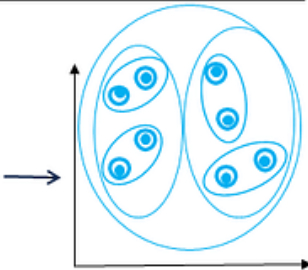
Each datapoint is treated as a cluster



Forming a cluster with the two closest datapoints



Forming a bigger cluster with the closest clusters.

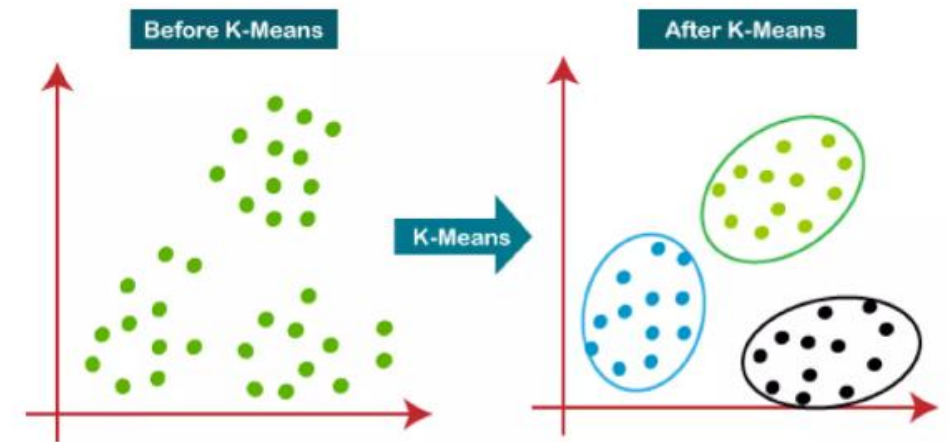


All clusters combined into a single cluster.

Reference:
<https://medium.com/@mbali.kalirane/hierarchical-clustering-vs-k-means-clustering-how-do-the-clustering-algorithms-differ-f0445e63519f>

K-means clustering algorithm 개요

- K-means clustering 알고리즘은 n 개의 인스턴스가 주어졌을 때, 이들을 K 개의 클러스터로 할당하는 알고리즘이다.
- 이때 클러스터 간의 거리는 최대한 멀어지도록 동시에 같은 클러스터 내 인스턴스 간의 거리는 최대한 가까워지도록 K 개의 클러스터를 구성하는 것이 핵심이다.
- K-means clustering 알고리즘의 구성
 - 임의의 두 인스턴스 간의 유사도 측정 함수
 - 각 클러스터의 centroid vector를 계산하는 함수



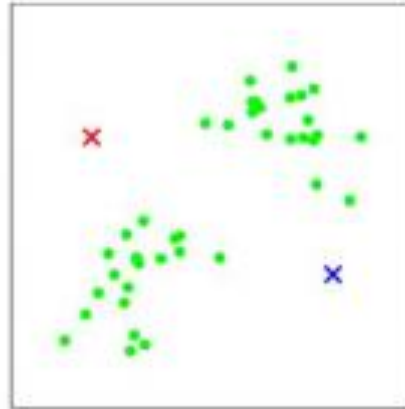
K-means clustering algorithm의 수행 절차

- ① N개의 인스턴스가 주어진다.
- ② N개의 인스턴스 중 K개를 임의로 선택하여 각 클러스터의 초기(initial) centroid vector로 지정한다.
- ③ N개의 인스턴스(centroid vector가 아닌) 각각에 대하여, 다음 작업을 수행한다.
 - 1) K개의 클러스터 내 centroid vector들과의 거리를 각각 계산한다.
 - 2) 거리가 가장 짧은 centroid vector를 갖는 클러스터에 해당 인스턴스를 할당한다.
- ④ 모든 클러스터들의 centroid vector를 갱신한다.
- ⑤ 단계 ③ ~ ④를 수렴(converge)할 때 까지 반복한다.
- ⑥ 클러스터 $C_1, \dots, C_K$ 를 최종적으로 반환한다.

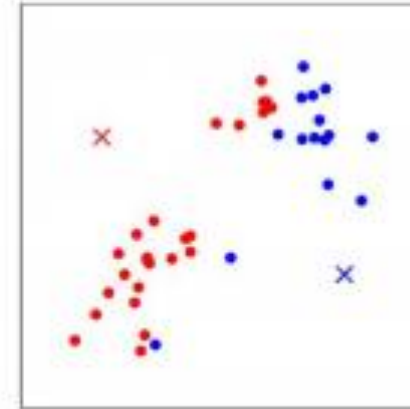
K-means clustering algorithm의 수행 절차



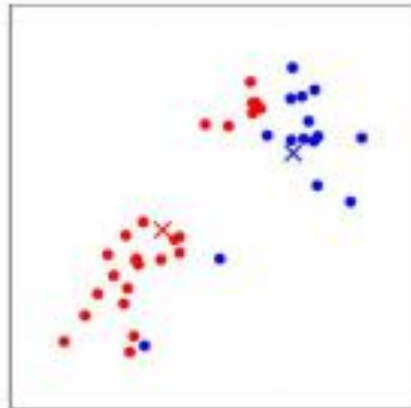
(a)



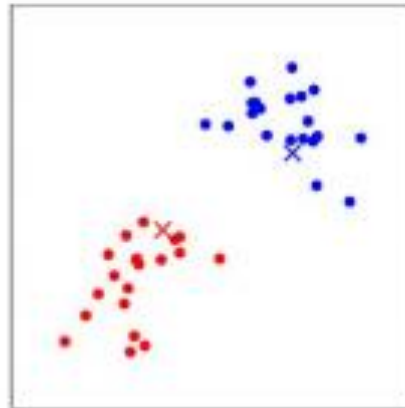
(b)



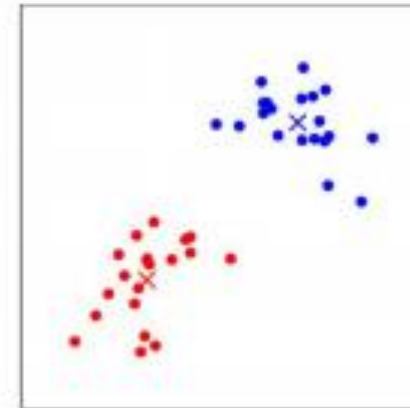
(c)



(d)



(e)



(f)

인스턴스 (Instance)

- D개의 실수 성분으로 구성된 D차원의 벡터

$$\vec{x}^{(i)} = [x_{i,1}, x_{i,2}, \dots, x_{i,D}]^T \in \mathbb{R}^D$$

- K-NN classification 알고리즘에서는 각각의 인스턴스가 어떤 클래스에 속하는지를 나타내는 label 정보가 training dataset에 포함되어 있다.

→ Supervised learning

- 그러나, K-Means clustering 알고리즘에서는 각 인스턴스가 어떤 클러스터에 속하는지를 나타내는 label 정보가 존재하지 않는다.

→ Unsupervised learning

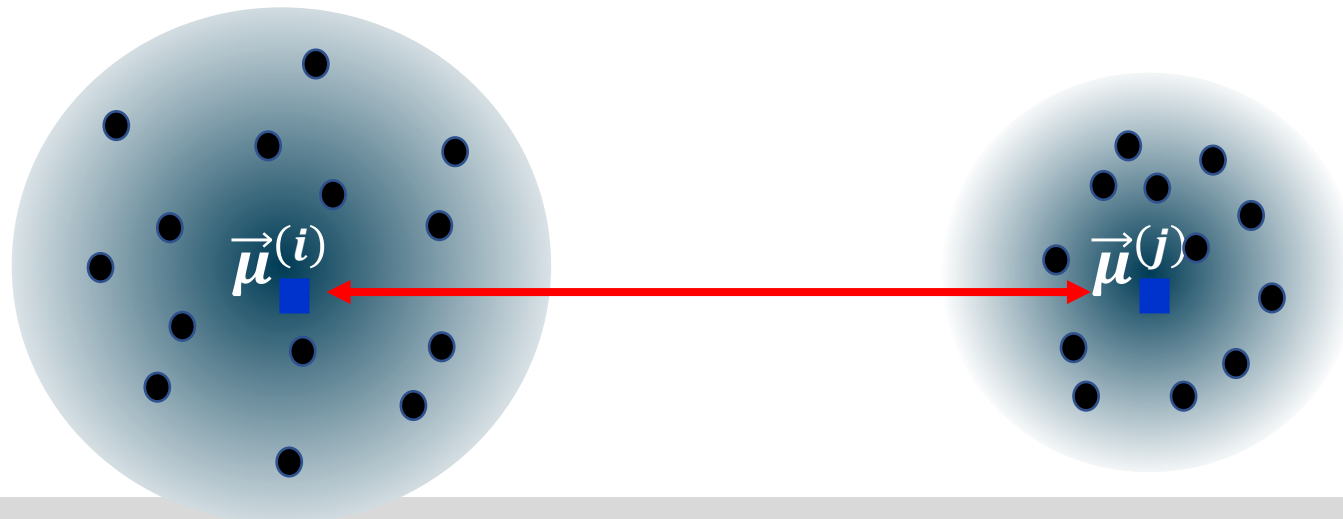
클러스터 (Cluster)

- 클러스터란 서로 유사한 특징을 갖는 인스턴스들의 집합을 의미한다.
- 클러스터링 알고리즘에서는 각 인스턴스에 라벨 정보가 포함되어 있지 않다.
- 따라서, 각 인스턴스 간의 유사도를 분석하여 유사도가 높은 인스턴스끼리 하나의 클러스터로 묶어주는 방법을 사용한다.
- K-means clustering algorithm 에서는 주어진 인스턴스들을 K개의 클러스터로 클러스터링하며, 이때 K는 사용자에게 의해서 사전에 정의되는 parameter이다.

클러스터 (Cluster)

- 좋은 클러스터의 조건

- 서로 다른 클러스터 간의 거리가 멀수록 좋다.
- 같은 클러스터 내 인스턴스들의 거리가 centroid vector를 기준으로 가까울수록 좋다.
 - 즉, 같은 클러스터 내 인스턴스들의 밀집도가 높을수록 좋다.



Centroid vector

- 두 클러스터 간의 거리를 계산하기 위해서, 각 클러스터마다 평균 벡터를 정의하는데, 이를 클러스터의 **centroid vector**라 한다.
- 클러스터 C_i 의 centroid vector
 - 클러스터 C_i 내에 포함된 모든 인스턴스들의 평균 벡터(average vector)이며, 아래와 같이 계산된다.

$$\vec{\mu}^{(i)} = \frac{1}{|C_i|} \sum_{\forall \vec{x} \in C_i} \vec{x}$$

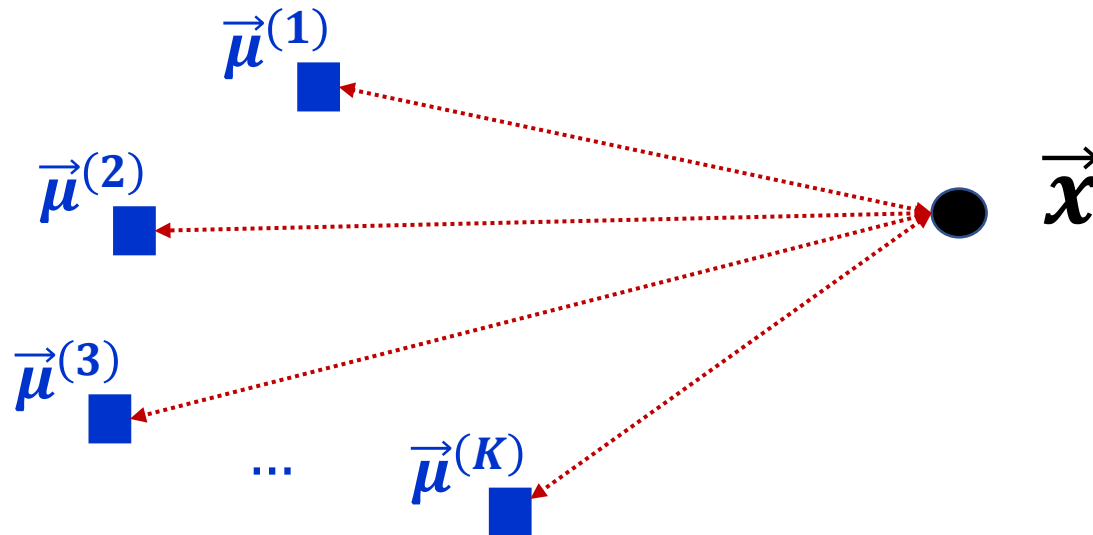
- 클러스터 내 인스턴스들이 D 차원이라면, centroid vector 역시 D 차원이다.

$$\vec{\mu}^{(i)} = [\mu_1^{(i)}, \mu_2^{(i)}, \dots, \mu_D^{(i)}]^T \in \mathbb{R}^D$$

인스턴스를 특정 클러스터에 할당하는 방법

- 인스턴스 $\vec{x}$ 를 K개의 클러스터 중 하나로 할당하기 위해서는...
 - 각 클러스터의 centroid vector와 인스턴스 $\vec{x}$ 간의 거리를 계산한다.
 - 거리가 가장 짧은 centroid vector를 갖는 클러스터에 인스턴스 $\vec{x}$ 를 할당한다.

$$Dist(\vec{x}, \vec{\mu}^{(j)}) \leq Dist(\vec{x}, \vec{\mu}^{(r)}), \quad \forall r \text{ s.t. } 1 \leq r \leq K, r \neq j$$



Cluster	Distance
C ₁	4.53
C ₂	2.06
C ₃	3.57
C ₄	6.02
C₅	1.87
C ₆	5.48

[Summary] 인스턴스와 클러스터 간의 거리 계산과 할당

- 인스턴스 $\vec{x}$ 와 i 번째 클러스터 C_i 간의 거리 계산

- $\vec{x}$ 와 i 번째 클러스터 C_i 의 centroid vector $\vec{\mu}^{(i)}$ 간의 거리를 계산한다.
- 거리 계산 방법(distance measure)은 Euclidean distance 이외의 다른 방법도 사용 가능

$$Dist(\vec{x}, \vec{\mu}^{(i)}) = \|\vec{x} - \vec{\mu}^{(i)}\|^2 = \sqrt{\sum_{j=1}^D (\vec{x}_j - \vec{\mu}_j^{(i)})^2}$$

- 인스턴스 $\vec{x}$ 를 클러스터로 할당하는 방법

- $\vec{x}$ 는 아래와 같이 centroid vector와의 거리가 최소인 클러스터 C_j 로 할당된다.

$$Dist(\vec{x}, \vec{\mu}^{(j)}) \leq Dist(\vec{x}, \vec{\mu}^{(r)}), \forall r \text{ s. t. } 1 \leq r \leq K, r \neq j$$

또는

$$\arg \min_j Dist(\vec{x}, \vec{\mu}^{(j)})$$

인스턴스 할당 후 클러스터의 centroid 갱신

- 인스턴스 $\vec{x}$ 가 j 번째 클러스터에 할당되었다고 가정하자.
- 그렇다면, j 번째 클러스터에는 새로운 인스턴스 $\vec{x}$ 가 추가된 상황이므로, j 번째 클러스터의 centroid vector인 $\vec{\mu}^{(j)}$ 또한 새롭게 업데이트된다.

$$\vec{\mu}^{(i)} = \frac{1}{|C_i|} \sum_{\forall \vec{x} \in C_i} \vec{x}$$

- 계산의 효율성(efficiency)을 위해서, centroid vector의 업데이트는 아래와 같이 incremental computation 방법으로도 구현할 수 있다.

```
nums[i] = nums[i] + 1;  
sum_ins[i] = sum_ins[i] + x;  
mu[i] = (1/nums[i])*sum_ins[i];
```

- **nums[i]**: i 번째 클러스터 내 인스턴스의 개수
- **sum_ins[i]**: i 번째 클러스터 내 모든 인스턴스 벡터들의 합
- **mu[i]**: i 번째 클러스터의 centroid vector

K-means clustering algorithm의 의사 코드

Set K centroid vectors randomly from n instances

inst_cluster[1,..., n] = -1

do

for $i = 1$ to n do

$$\gamma_{ij} = \begin{cases} 1 & \text{if } j = \arg \min_j \text{Dist}(\vec{x}^{(i)}, \vec{\mu}^{(j)}) \\ 0 & \text{otherwise} \end{cases}$$

inst_cluster[i] = j

end for

for $j = 1$ to K do

$$n_j = \sum_{i=1}^n \gamma_{ij}$$

$$\vec{\mu}^{(j)} = \frac{1}{n_j} \sum_{i=1}^n \gamma_{ij} \vec{x}^{(i)}$$

end

until (*termination condition is satisfied*)

return inst_cluster

• i 번째 인스턴스와 거리가 가장 가까운 centroid vector가 $\vec{\mu}^{(j)}$ 이면, γ_{ij} 를 1로 지정한다.

• 즉, γ_{ij} 는 i 번째 인스턴스가 j 번째 클러스터에 속하면 1, 아니면 0을 갖는 Boolean variable이다.

• 각각의 클러스터에 대하여, n_j 는 j 번째 클러스터 내 인스턴스의 개수를 의미한다.

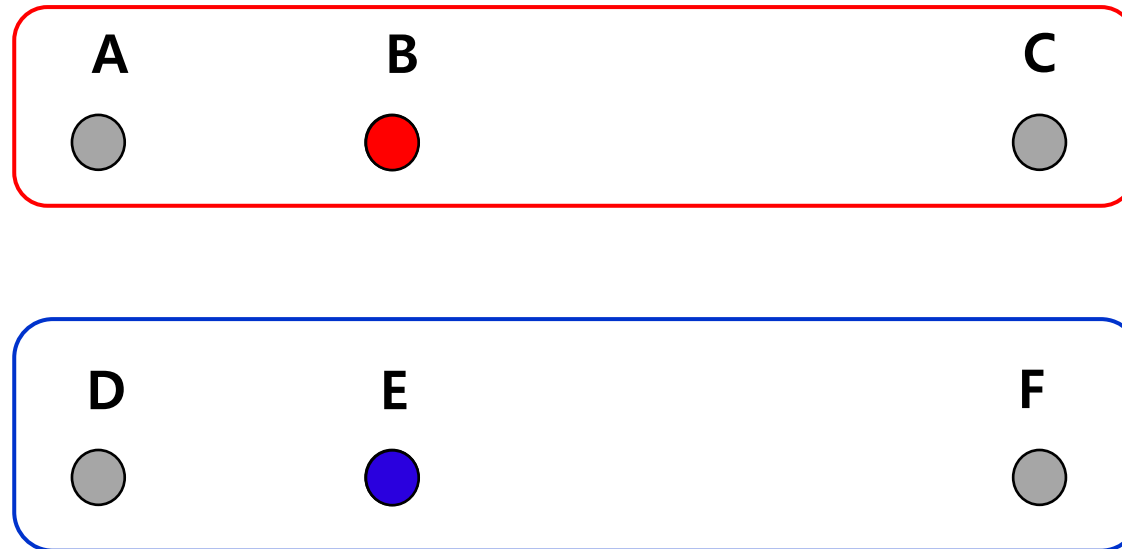
• j 번째 클러스터의 centroid vector를 업데이트한다.

• 이때, $\vec{x}^{(i)}$ 가 j 번째 클러스터에 포함되어 있으면, $\gamma_{ij} = 1$ 이므로, $\vec{x}^{(i)}$ 가 평균 계산에 반영된다.

• 만약 $\vec{x}^{(i)}$ 가 j 번째 클러스터에 포함되어 있지 않다면, $\gamma_{ij} = 0$ 이므로, $\vec{x}^{(i)}$ 가 평균 계산에 반영되지 않는다.

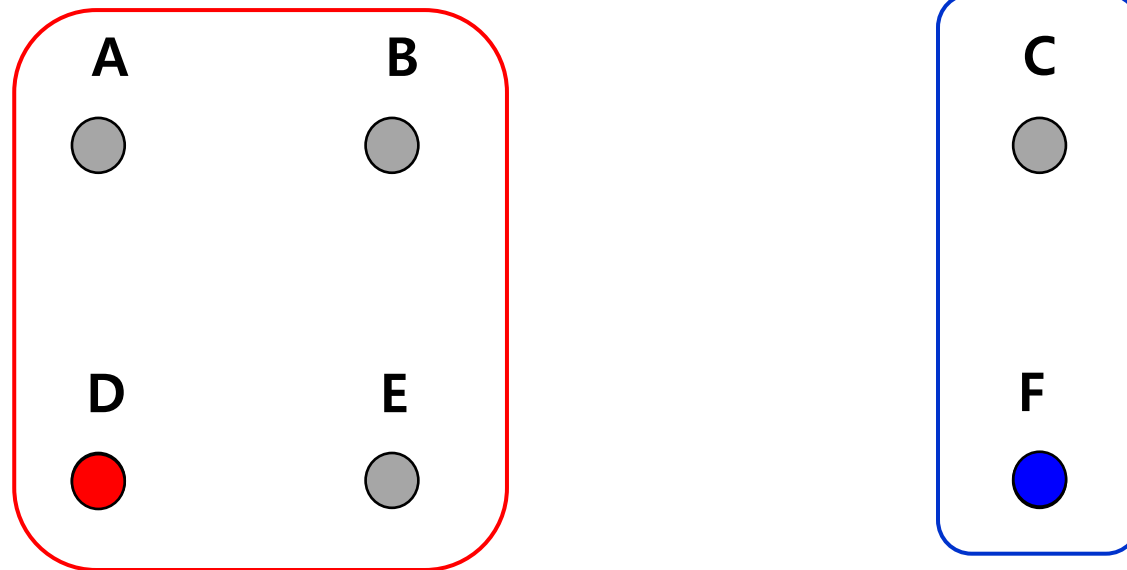
초기 centroid vector 지정 방법과 클러스터링 결과와의 연관성

- $K = 2$ 이고, 인스턴스의 개수가 6인 아래 예제를 고려해 보자.
- 만약 B와 E가 초기 centroid vector로 지정된다면, 최종 클러스터링 결과는 $C_1 = \{A, B, C\}$, $C_2 = \{D, E, F\}$ 가 된다.



초기 centroid vector 지정 방법과 클러스터링 결과와의 연관성

- 반면에, D와 F가 각각 centroid vector로 초기화된다면, 최종 클러스터링 결과는 아래와 같이 $C_1 = \{A, B, D, E\}$, $C_2 = \{C, F\}$ 가 된다.
- 결과적으로, 초기 centroid vector를 지정하는 방법에 따라 클러스터링 결과가 달라질 수 있다.



K-means clustering algorithm의 종료 조건

- 앞서 설명한 K-mean clustering algorithm의 의사 코드를 살펴보면 **do ~ until (termination condition is satisfied)**로 표기된 부분이 존재한다.
- 즉, K-means algorithm은 특정 종료 조건을 만족할 때까지 반복하여 수행된다.
- 이때, 종료 조건은 다양한 방법으로 구현할 수 있으며, 대표적인 방법은 다음과 같다.
 - 최대 반복 횟수(The maximum number of iterations)를 초과하는 경우
 - 각각의 클러스터의 centroid vector가 더 이상 변화하지 않거나 변화의 정도가 매우 작을 경우
 - 즉, 클러스터 내 상태(state)가 더 이상 유의미한 변화를 보이지 않는 경우

K-Means clustering algorithm의 구현

K-means clustering algorithm의 구현

- K-means clustering algorithm의 기능을 구현하는 방법을 세부적으로 이해하기 위해 C (또는 C++)로 구현해 보자.
 - ① LMS에서 “kmeans.cpp” 파일을 다운로드 받는다.
 - ② “kmeans.cpp” 파일의 주요 함수와 코드의 기능을 주석을 참고하여 분석한다.
 - ③ 각 함수 내 모든 빈 칸을 직접 완성해본다.

거리 측정 함수의 구현

- 거리 측정 함수는 유클리드 거리 함수를 사용한다.

$$Dist(\vec{x}, \vec{\mu}^{(i)}) = \|\vec{x} - \vec{\mu}^{(i)}\|^2 = \sqrt{\sum_{j=1}^D (\vec{x}_j - \vec{\mu}_j^{(i)})^2}$$

```
// 거리 측정 함수 (Euclidean distance)
float distance(float a[], float b[]) {
    float sum = 0.0;
    float diff = 0.0;

    return sqrt(sum);
}
```

초기 centroid vector 선택

- 주어진 인스턴스들 중에서 K개의 인스턴스를 임의로 선택하여 centroid vector로 지정한다.
- 구현의 편의를 위해 본 예제에서는 random selection 기능은 구현하지 않고, seed_cents[] 배열에 지정된 인스턴스들을 centroid vector로 사용하기로 한다.

```
int seed_cents[K] = { 1, 10, 17 };
for (int k = 0; k < K; k++) {
    int idx = seed_cents[k];
    
}
```

인스턴스와 클러스터 간 거리 측정 및 클러스터 할당

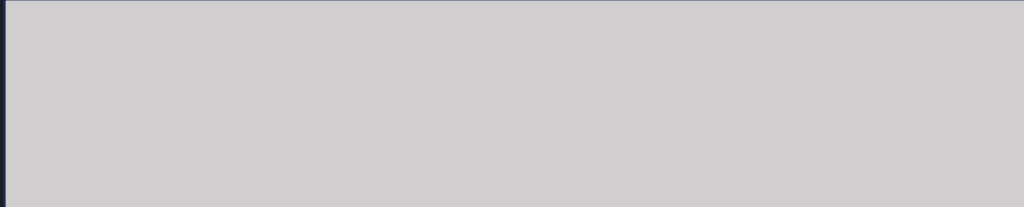
- 각각의 인스턴스에 대하여 가장 가까운 centroid vector를 갖는 클러스터로 해당 인스턴스를 할당한다.

```
// 1. 각 인스턴스에 가장 가까운 centroid를 갖는 클러스터로 할당
for (int i = 0; i < NUM_INSTANCE; i++) {
    float min_dist = FLT_MAX;
    int min_idx = 0;
    
    clusters[i] = min_idx; // i번째 인스턴스의 클러스터를 min_idx로 할당
}
```

클러스터의 Centroid vector를 갱신

- 클러스터의 centroid vector는 다음과 같은 순서로 갱신한다.
 - ① 각 클러스터에 속하는 모든 인스턴스들을 하나로 더한다.
 - ② 위의 합한 결과를 해당 클러스터에 속하는 인스턴스의 개수로 나눈다.
 - ③ 해당 클러스터의 centroid vector를 새롭게 계산한 결과로 업데이트한다.

```
// 2. 각 클러스터의 centroid를 계산
float new_centroids[K][D] = { 0 };
int counts[K] = { 0 };
// 2-1. 각 클러스터에 속하는 인스턴스들을 모두 합한다.
for (int i = 0; i < NUM_INSTANCE; i++) {
    int cluster = clusters[i];
```

- ```
// 2-2. 각 클러스터에서 합한 인스턴스들을 클래스 내 인스턴스의 갯수로 나눈다(평균 계산).
for (int k = 0; k < K; k++) {
 
}

```

- ```
// 2-3. 위에서 계산한 클러스터 별 centroid vector로 새롭게 업데이트한다.
for (int k = 0; k < K; k++) {
    
}
```

실제 구현 결과 확인하기

Instances and these clusters:

```
Instance 0 : Cluster 0
Instance 1 : Cluster 0
Instance 2 : Cluster 0
Instance 3 : Cluster 1
Instance 4 : Cluster 2
Instance 5 : Cluster 0
Instance 6 : Cluster 2
Instance 7 : Cluster 0
Instance 8 : Cluster 0
Instance 9 : Cluster 0
Instance 10 : Cluster 1
Instance 11 : Cluster 0
Instance 12 : Cluster 2
Instance 13 : Cluster 2
Instance 14 : Cluster 2
Instance 15 : Cluster 0
Instance 16 : Cluster 0
Instance 17 : Cluster 2
Instance 18 : Cluster 2
Instance 19 : Cluster 2
```

Final centroids:

```
Centroid vector of Cluster 0: [ 6.57  5.19  2.69  5.22  6.31  4.66  6.58  5.50  4.99  6.93 ]
Centroid vector of Cluster 1: [ 5.77  1.39  6.95  2.22  2.94  3.89  3.50  7.10  4.76  9.08 ]
Centroid vector of Cluster 2: [ 5.93  3.94  4.79  4.64  5.34  5.09  3.93  3.66  4.09  2.43 ]
```

Q & A